Labo 13 — Réponses commentées

Chaque réponse suit le même schéma : la réponse, le mécanisme, la nuance ou le piège, un exemple vérifiable au terminal.

Question 1 — `host not found in upstream "api"`

Réponse. Au labo 07, le DNS du réseau Podman donnait un nom à chaque **conteneur**. Dans le cluster, un Pod n'a **pas** de nom DNS : seuls les **Services** en ont. nginx résout `api` au démarrage (labo 07), ne trouve rien, et sort en erreur ; le kubelet le relance, d'où `CrashLoopBackOff`. Il faut créer le Service `api` (selector `app: api`, port 8080) : CoreDNS crée alors l'entrée, et à la relance suivante du backoff nginx démarre — `web` repart seul parce que `restartPolicy: Always` fait réessayer indéfiniment.

Pourquoi. Le nommage est une responsabilité du Service, pas du Pod : les Pods sont trop éphémères pour mériter un nom.

Nuance. Dans un manifeste complet, on applique le Service **avant** ou **avec** le Deployment du front ; l'ordre dans un même `apply -f dossier/` est alphabétique, d'où l'habitude de tout mettre dans un fichier ou de laisser le backoff faire.

Exemple.

```
kubectl apply -f service-api.yaml && kubectl rollout status deployment/web --timeout=120s   # successfully rolled out
```

Question 2 — IP de cluster, DNS, EndpointSlices

Réponse. L'**IP de cluster** (`10.43.x.x`) est une adresse virtuelle attribuée par l'API server à la création du Service, stable tant qu'il existe. Le **nom DNS** `api.lab13.svc.cluster.local` est maintenu par CoreDNS, qui observe les Services. Les **EndpointSlices** sont la liste des `IP:port` des Pods qui portent le label du selector **et** sont prêts, maintenue par le contrôleur d'endpoints. `ping` échoue parce que l'IP n'existe sur aucune interface : les règles de kube-proxy ne traduisent que les connexions TCP/UDP vers un port du Service, pas l'ICMP.

Pourquoi. Le Service est une abstraction de NAT distribuée, pas une machine.

Nuance. Avec kube-proxy en mode IPVS (ou Cilium sans kube-proxy), l'IP peut répondre au `ping` — ne vous fiez pas au `ping` pour diagnostiquer un Service.

Exemple.

```
kubectl get svc api -o jsonpath='{.spec.clusterIP}' ; kubectl get endpointslices -l kubernetes.io/service-name=api
```

Question 3 — Les endpoints suivent les Pods

Réponse. Après le passage à 3 réplicas : **trois** adresses (dès que le troisième Pod est prêt). Après la suppression d'un Pod : l'adresse disparaît **immédiatement**, puis celle du Pod de remplacement apparaît quand il est prêt (quelques secondes à deux adresses). Après la readiness cassée : l'adresse de ce Pod est **retirée**, deux restent, le Pod continue de tourner. Le client qui appelait

`api:8080` n'a rien vu, sauf une connexion coupée s'il était précisément sur le Pod supprimé au mauvais moment.

Pourquoi. Le contrôleur d'endpoints réagit aux changements de Pods (création, suppression, condition `Ready`) et kube-proxy met à jour ses règles en moins d'une seconde.

Nuance. Entre le moment où un Pod reçoit `SIGTERM` et son retrait des endpoints, quelques requêtes peuvent encore lui arriver : c'est pourquoi une application doit gérer l'arrêt propre (labo 03) et pourquoi on ajoute parfois un `preStop` de quelques secondes.

Exemple.

```
kubectl get endpointslices -l kubernetes.io/service-name=api -o
jsonpath='{.items[0].endpoints[*].addresses[0]}' ; echo
```

Question 4 — Un Service sans endpoints

Réponse. Soit le **selector** ne correspond à aucun label de Pod (`app: apis`, `app: backend`, une faute de frappe), soit les Pods ne sont **pas prêts** — mais ici ils sont `1/1`, donc c'est le selector. `kubectl get pods --show-labels` à comparer avec `kubectl get svc api -o jsonpath='{.spec.selector}'` tranche. Correction : corriger le selector (ou les labels du template) et réappliquer ; pour le cas readiness, corriger la sonde.

Pourquoi. L'EndpointSlice est calculé à partir du selector ; aucune correspondance, aucune adresse, et Traefik, qui route vers les endpoints, n'a rien vers quoi router : `503`.

Nuance. `kubectl describe svc api` montre `Endpoints: <none>` en clair ; c'est la ligne à chercher.

Exemple.

```
kubectl get svc api -o jsonpath='{.spec.selector}' ; echo ; kubectl get pods --show-labels |
head -3
```

Question 5 — Trois façons d'exposer le front

Réponse. ClusterIP : aucune URL depuis l'hôte, seulement `kubectl port-forward svc/web 18080:80` puis `http://localhost:18080`. NodePort : `http://<ip-du-nœud>:30080` (et souvent `http://localhost:30080` depuis WSL). LoadBalancer : `http://<EXTERNAL-IP>:8081`, où l'`EXTERNAL-IP` est l'IP du nœud parce que **ServiceLB** de k3s lance un Pod `svclb-web-lb` qui ouvre le port sur le nœud — Kubernetes nu laisserait `<pending>` faute de fournisseur. Un second LoadBalancer sur le port 80 resterait `<pending>` parce que ServiceLB ne peut pas ouvrir deux fois le même port sur le même nœud, et que Traefik l'occupe déjà.

Pourquoi. ServiceLB est un équilibreur « de pauvre » : un port hôte par Service, sur chaque nœud.

Nuance. Sur le cloud, le même YAML commande un vrai équilibreur avec sa propre IP ; les ports ne se disputent plus, mais chacun coûte.

Exemple.

```
kubectl get pods -n kube-system -l svccontroller.k3s.cattle.io/svcname=web-lb -o name   #
pod/svclb-web-lb-…
```

Question 6 — Le nom court ne marche que dans le namespace

Réponse. Le `/etc/resolv.conf` d'un Pod contient `search lab13.svc.cluster.local svc.cluster.local cluster.local` et `nameserver 10.43.0.10` (CoreDNS). Depuis `lab13`, `api` devient `api.lab13.svc.cluster.local` par le premier suffixe. Depuis `default`, le premier suffixe est `default.svc.cluster.local` : `api.default.svc.cluster.local` n'existe pas, puis `api.svc.cluster.local` non plus. Il faut écrire `api.lab13` (le deuxième suffixe complète en `api.lab13.svc.cluster.local`) ou le nom complet.

Pourquoi. Le `search` est généré par le kubelet avec le namespace du Pod ; `ndots:5` fait tenter les suffixes avant une résolution directe.

Nuance. Appeler un Service d'un autre namespace est permis par défaut ; une NetworkPolicy (labo 16) peut l'interdire.

Exemple.

```
kubectl exec deploy/web -- cat /etc/resolv.conf
kubectl run -n default -it --rm t --image=docker.io/library/alpine -- wget -qO-
http://api.lab13:8080/actuator/health
```

Question 7 — Connexions, pas requêtes

Réponse. kube-proxy choisit un Pod **à l'ouverture de chaque connexion TCP** et ne regarde jamais le contenu. Un client HTTP/1.1 en *keep-alive* (ou un pool JDBC) ouvre peu de connexions et les garde : toutes ses requêtes vont au **même** Pod, et trois réplicas ne se partagent la charge que s'il y a assez de clients. L'Ingress (Traefik), lui, termine la connexion du client et ouvre les siennes vers les Pods : il répartit **par requête HTTP**, et peut même faire du *sticky* ou du pondéré.

Pourquoi. Le Service est un mécanisme de niveau 4 ; l'équilibrage de niveau 7 demande un proxy qui comprend HTTP.

Nuance. Pour une base de données (connexions longues), c'est acceptable et même souhaité ; pour une API appelée par un seul front, c'est l'argument pour passer par l'Ingress plutôt que par le proxy nginx du Pod `web`.

Exemple.

```
for i in 1 2 3 4 ; do kubectl exec deploy/web -- wget -qO- http://api:8080/ ; done   # chaque
wget = nouvelle connexion = Pod au hasard
```

Question 8 — `port` et `targetPort`

Réponse. `port: 80` est le port **du Service** (celui qu'on appelle : `api:80`, soit `http://api/`) ; `targetPort: 8080` est le port **du conteneur** vers lequel kube-proxy traduit. `api:8080` n'est servi par personne, d'où `Connection refused`. nginx doit écrire `proxy_pass http://api/;` (ou `http://api:80/`).

Pourquoi. Le Service découple le port exposé du port réel, ce qui permet de changer l'un sans l'autre — et de donner à tous les Services un port « propre ».

Nuance. `targetPort` peut être un **nom** de port du conteneur (`targetPort: http` si le conteneur déclare `name: http`), ce qui résiste à un changement de numéro dans l'image.

Exemple.

```
kubectl get svc api -o jsonpath='{.spec.ports[0].port} -> {.spec.ports[0].targetPort}' ; echo
```

Question 9 — Le chemin d'une requête

Réponse. Navigateur Windows → forwarding WSL → port **80** du nœud, ouvert par le Pod `svclb-traefik` (ServiceLB) → Service `traefik` → Pod **Traefik**, qui lit l'Ingress `web` (`/` → Service `web`) → Service `web` → Pod `web` (**nginx**), dont la configuration relaie `/api/` → Service `api` → un Pod `api`, qui répond `{"status":"UP"}`. Deux reverse proxies : Traefik puis nginx. Simplification : ajouter à l'Ingress une règle `path: /api` → Service `api` port 8080 (avec une réécriture du préfixe), et retirer le `location /api/` de nginx — Traefik route directement.

Pourquoi. Chaque composant ne connaît que le suivant : c'est ce qui rend chaque maillon remplaçable.

Nuance. La réécriture `/api/x` → `/x` se fait chez Traefik par un *middleware* (`stripPrefix`), chez ingress-nginx par une annotation `rewrite-target` : la syntaxe dépend du controller, pas de l'objet Ingress standard.

Exemple.

```
kubectl describe ingress web | grep -A4 Rules
```

Question 10 — Ingress et Ingress controller

Réponse. L'`Ingress` est un objet de l'API : des règles de routage, rien de plus. L'*Ingress controller* est un **programme** (Traefik, nginx…) déployé dans le cluster, qui observe ces objets et configure son reverse proxy en conséquence. Sans controller, un Ingress est accepté, stocké, et **sans effet** : pas d'`ADDRESS`, aucun trafic. Un `503` de Traefik signifie que Traefik a bien reçu la requête, a trouvé la règle, mais que le Service visé n'a **aucun endpoint prêt** : le problème est en aval (Pods, readiness, selector), Traefik n'est que le messager.

Pourquoi. Kubernetes définit l'interface, pas l'implémentation ; même modèle que CNI (réseau) et CSI (stockage).

Nuance. Un cluster peut avoir plusieurs controllers ; `spec.ingressClassName` (ou la classe par défaut, `traefik` chez k3s) dit lequel prend l'objet.

Exemple.

```
kubectl get ingressclass ; kubectl get ingress web -o jsonpath='{.spec.ingressClassName}'
```

Question 11 — L'Ingress sans `ADDRESS` sur EKS

Réponse. Il manque un **Ingress controller** : k3s en livre un (Traefik), EKS non. Les objets Ingress sont stockés mais personne ne les lit ; l'`ADDRESS` reste vide. Il faut installer un controller (ingress-nginx, Traefik, ou le AWS Load Balancer Controller) et, selon le cas, renseigner `ingressClassName`. Cela « marchait » sur k3s parce que Traefik était déjà là, avec sa classe par défaut.

Pourquoi. Les distributions choisissent ce qu'elles embarquent ; un cluster managé laisse ce choix à l'équipe plateforme.

Nuance. Sur EKS, le Service `LoadBalancer` fonctionne sans rien installer parce que l'intégration cloud **est** fournie (le cloud-controller-manager) : c'est l'inverse de k3s, où c'est ServiceLB qui

simule.

Exemple.

```
kubectl get pods -A | grep -i -E 'ingress|traefik'   # vide = pas de controller
```

Question 12 — `/api/` par nginx ou par l'Ingress

Réponse. Même origine = pas de CORS, une seule URL à configurer, le front ignore où vit l'API (labo 07). Dans notre stack, le routage `/api/` peut être fait par **nginx** (le `location /api/` du Pod `web`) ou par l'**Ingress** (une règle `path: /api`). Confier à l'Ingress : un seul proxy au lieu de deux, l'image `web` n'a plus besoin de connaître le nom `api` (donc plus de crash au démarrage), l'équilibrage par requête de Traefik vers les Pods `api`, le TLS et les en-têtes au même endroit, et un front réduit à des fichiers statiques.

Pourquoi. Le routage est une responsabilité de la plateforme ; le front statique n'a pas à le porter.

Nuance. Le proxy dans nginx reste utile sur le poste (Compose, labo 09) où il n'y a pas d'Ingress : certaines équipes gardent les deux configurations.

Exemple.

```
- path: /api
  pathType: Prefix
  backend: { service: { name: api, port: { number: 8080 } } }
```

Question 13 — Joindre la base depuis Windows

Réponse. Exposition croissante : `port-forward` (un tunnel chiffré par l'API server, valable le temps de la commande, visible par vous seul) < NodePort (un port ouvert sur tous les nœuds, pour tout le réseau) < LoadBalancer (une IP dédiée, publique sur le cloud). En production, seule la première est acceptable, et encore, pour un diagnostic ponctuel avec les droits RBAC adéquats (labo 16) : une base n'a jamais de port vers l'extérieur. Ce qui manque encore : un **Service** `db` (ClusterIP, ou *headless* pour un StatefulSet, labo 15) pour que `db` ait un nom dans le cluster.

Pourquoi. Chaque type de Service élargit la surface d'exposition ; le besoin d'un humain n'est pas celui de l'application.

Nuance. En entreprise, l'accès humain aux bases passe par un bastion ou un outil d'audit (Teleport, Boundary), jamais par un port de nœud.

Exemple.

```
kubectl port-forward svc/db 5432:5432   # puis DBeaver sur localhost:5432, le temps de la
session
```